

SD Dictionaries - Using and Creating

How the query processor reads a dictionary, the system dictionaries, and creating one.

This page continues SD Dictionaries - Structure.

How the query processor uses the dictionary

When a query command runs — `LIST STOCK DESCRIPTION QTY` — the query processor opens the dictionary of `STOCK` and looks up each name the command mentions.

NAME RESOLUTION IN THE DICTIONARY

A name is looked up **as typed, then lower case, then upper case**, the same resolution the VOC uses. If the dictionary has no record by that name, the query reports *"field name not found"* and stops.

If the name is not in the file's own dictionary, SD looks in the account VOC for a phrase of the same name — a `PH` record that expands to one or more dictionary field names. This is how saved selection clauses work: `list stock with qty > 100` can use a phrase `high.qty` holding `WITH QTY > 100` that is stored in the VOC, not the dictionary.

WHAT THE QUERY PROCESSOR READS

For each field name in the command, the query processor reads the dictionary record and extracts:

1. **The field number** (field 2 for D, A and S; the expression for I and C) — where to get the data.
2. **The conversion code** (field 3) — how to transform it for display.
3. **The display name** (field 4) — what to head the column.
4. **The format** (field 5) — how wide and how justified.
5. **The S/M flag** (field 6) — whether to expect multiple values.
6. **The association** (field 7) — which fields to print side by side.

For an I-type, it compiles the expression (if the stamp has changed), executes it for each record, and uses the result as the field value.

THE DEFAULT FORMAT

If field 5 is empty, the query processor picks a default — ten characters, left-justified for strings, right-justified for numbers. The column heading defaults to the field name in upper case.

The system dictionaries

Three dictionaries ship in `@SDSYS` and are pointed at by every account's VOC:

VOC ID	DICTIONARY OF
<code>voc</code>	the VOC itself (<code>@SDSYS/voc.dic</code>)
<code>dict.dic</code>	the dictionary file (<code>@SDSYS/dict.dic</code>)
<code>newvoc</code>	the NEWVOC template

The `voc.dic` dictionary is what `dict voc` reads. It has records for `@ID`, `F1` through `F5`, `TYPE`, `FTYPE`, `DESC`, `DATA.NAME`, `DICT.NAME`, `NAME`, `PROCESSOR`, `DISPATCH` and `IS.REMOTE` — the fields a VOC record has. Several of these are I-types that compute their display from other fields, which is why `dict voc` shows a type column that reads `V`, `K`, `F` and so on rather than the raw field 1.

Creating and editing dictionaries

ED — THE EDITOR

`ED DICT file.name field.name` is the way to create or modify a dictionary record by hand. The editor writes the record when you file it, and if it is an I-type, the stamp in field 15 is cleared so the next query recompiles it.

CREATE.FILE — THE DICTIONARY IS CREATED WITH THE FILE

When `CREATE.FILE` creates a data file, it creates the dictionary with it. The dictionary starts empty — no field records — and you add them with `ED` or by copying from another file's dictionary.

COPY — COPY DICTIONARY RECORDS

```
copy from dict src.file to dict tgt.file field.name
```

`copy` can copy dictionary records between files, which is the way to clone a field definition from one file's dictionary to another.

GENERATE — CREATE INCLUDE FILES FROM DICTIONARIES

`GENERATE` reads a dictionary and writes a BASIC include file (`.H`) that defines field numbers as constants. This is how a program avoids hard-coding field numbers: `$INCLUDE file.H` brings in `F1`, `F2` and the rest, and if a field number changes, the include file is regenerated and every program that includes it picks up the change on recompile.

What is not here

There is no verb that lists dictionary records by type. `DICT file.name` displays the records, and `LIST DICT file.name` runs the query processor over them, but there is no `listd` or `listdict` verb. `ED DICT file name` edits one record; `DICT file` displays several.

L-types are not in a stock dictionary. Type `L` is handled by the query processor and there is no shipped example. It exists for applications that need cross-file joins in a query.

The `C` type is rare. Stock dictionaries use `D` for direct fields and `I` for computed ones. `C` exists for compatibility and is functionally a subset of `I` — use `I` for new computed fields.

See also

SD Dictionaries - Conversions and Formatting · SD VOC - Structure and Usage · SD TCL - The Query Processor · SD Basic - File Handling · SD Basic - Dynamic Arrays.